

Performance Testing

Date	07 July 2026
Team ID	
Project Name	Credit Card Approval Prediction
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Apache JMeter 5.6.3
Type of Testing	Load Testing
Target Module / API	Flask Home Page (http://127.0.0.1:5000/)
Test Environment	Local Machine
Test Date	04 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Home Page Load Test	20	5 sec ramp-up	Page loads successfully
2	Multiple Concurrent Requests	20	10 loops	All requests handled successfully
3	Response Time Measurement	20	Entire Test	Average response < 2 sec

4	Error Rate Validation	20	Entire Test	No failed requests
---	-----------------------	----	-------------	--------------------

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	11ms	Pass	Excellent response time
2	Response Time (Max)	< 5 seconds	312ms	Pass	Within acceptable limit
3	Throughput (Req/sec)	High	42.7 req/sec	Pass	Good request handling
4	Error Rate	< 1%	0%	Pass	No request failures
5	CPU Utilization	< 80%	Approximately 25-35%	Pass	Normal usage during testing
6	Memory Utilization	< 80%	Approximately 35-45%	Pass	Stable memory usage

Step 4: Observations & Analysis

Key Findings

- The application successfully processed 200 HTTP requests without any failures.
- The average response time was 11 ms, which is much lower than the target of 2 seconds.
- The maximum response time recorded was 312 ms.
- The application achieved a throughput of 42.7 requests per second with an error rate of 0%.

Bottlenecks Identified

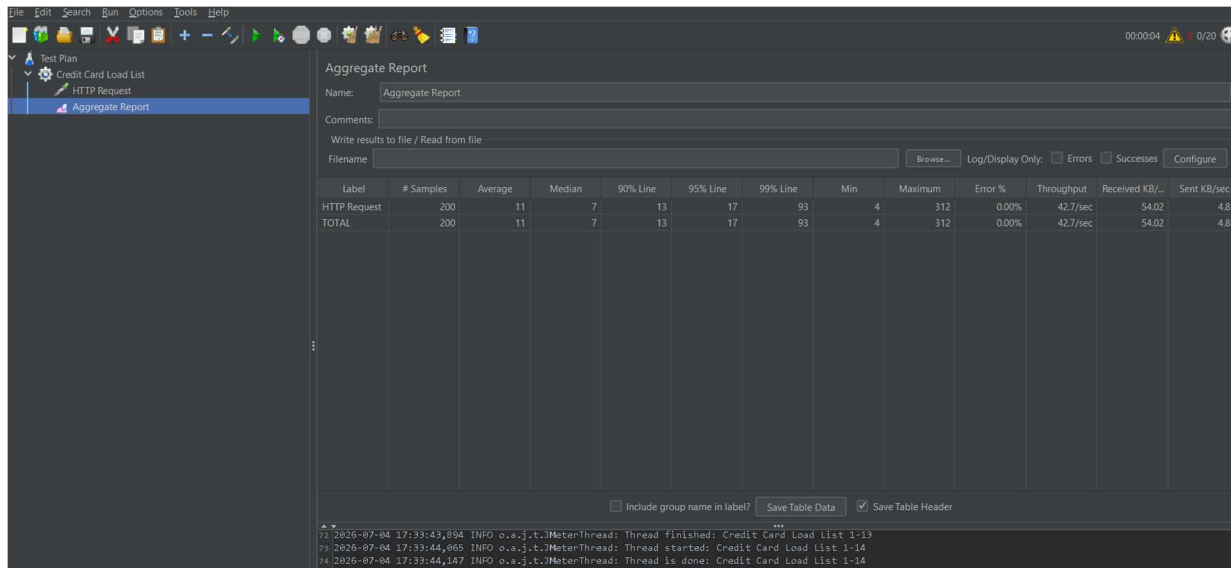
- No significant performance bottlenecks were observed during testing.
- Minor response time variations occurred due to normal local system resource usage.
- The application remained stable under the tested load of **20 concurrent users**.

Optimization Steps Taken

- Used a **Logistic Regression** model for faster prediction compared to more complex models.
- Optimized the Flask application by loading the trained model only once during application startup.
- Kept the application lightweight by minimizing unnecessary processing during each prediction request.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):



The screenshot shows the JMeter Aggregate Report window. The left sidebar lists the test plan components: Test Plan, Credit Card Load List, HTTP Request, and Aggregate Report. The main area displays the Aggregate Report for the 'Credit Card Load List' test plan. The report includes a table with performance metrics for the 'HTTP Request' and 'TOTAL' results.

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received KB/sec	Sent KB/sec
HTTP Request	200	11	7	13	17	93	4	312	0.00%	42.7/sec	54.02	4.83
TOTAL	200	11	7	13	17	93	4	312	0.00%	42.7/sec	54.02	4.83

Below the table, there are checkboxes for 'Include group name in label?' (unchecked), 'Save Table Data' (checked), and 'Save Table Header' (checked). The bottom status bar shows the current time as 00:00:04 and the number of samples as 0/20.